

Implementing Graph Analytic Algorithms on Relational Databases

1st Liangqi Chen

University of New South Wales

Sydney, Australia

z5438932@ad.unsw.edu.au

Code: <https://github.com/goblin-high-priest/GraphSQL>

Abstract—Graph analytics is becoming increasingly important, supporting many key applications such as social network analysis and machine learning. Since most graph data is stored in relational databases, it is natural to consider performing graph analytics directly within the relational database system. In this report, I implement several graph analytics algorithms on an RDBMS, which has been studied extensively over the past decades and is recognized for its robustness in managing and querying data.

Index Terms—PageRank, HITS, Topological Sort, Connected Component, Bellman Ford, Keyword Search, BSEG, GRAPHiQL

I. INTRODUCTION

A. Motivation

Graph analytics has become a routine workload in social networks, recommendation, transportation, the web, and enterprise scenarios. In practice, however, most large graphs are first collected and maintained inside relational database systems. Repeatedly exporting data to external graph engines introduces ETL overheads, stale snapshots, and governance gaps. In this report, I ask a simple but practical question: *can we run a useful set of graph algorithms directly and efficiently inside an RDBMS?* Prior work indicates that many graph workloads can be expressed as SQL dominated by scans, joins, and aggregates; modern column stores further make these patterns efficient through vectorization and compression [1], [2]. At the same time, writing graph logic in plain SQL is often verbose and hard to maintain, which motivates language support and execution patterns tailored to graphs [2], [3].

From a system viewpoint, I see three down-to-earth reasons to bring algorithms to the data. First, ACID, security, lineage, and backups already live in the RDBMS; staying in place keeps one source of truth and reduces moving parts. Second, iterative analytics can reuse mature optimizers and parallel execution without maintaining a separate stack [1]. Third, graph queries often mix with relational predicates (time windows, user segments, business rules), where pushing filters into SQL is both natural and efficient. The main challenge is the mismatch between the set-at-a-time relational style and the frontier-expansion nature of many graph algorithms.

I therefore take a deliberately pragmatic stance. I keep the schema minimal (normalized vertices/edges), avoid vendor-

specific extensions, and rely only on standard SQL features that are widely available (recursive WITH, window functions, MERGE/INSERT. . . ON CONFLICT). This choice is not only for portability; it also makes the implementations easier to audit, reproduce, and teach. In many organizations, database administrators prefer solutions that respect existing governance and monitoring; running inside the RDBMS reduces the number of systems to deploy and the risk of data drift between copies.

The algorithms I implement are intentionally representative. *PageRank* captures linear-algebra-style iterative updates; *HITS* adds alternating normalization; *Topological Sort* exercises partial-order processing; *Connected Components* is a propagation-style traversal; and *Bellman-Ford* (SSSP) expresses bulk relaxations. Together they cover common execution patterns (scan-join-aggregate, frontier expansion, and fixed points) that also underlie higher-level tasks such as keyword search and community queries. If these basics run acceptably in SQL, more complex tasks can be built as compositions rather than entirely new engines.

Finally, I adopt a simple performance mental model: total time $\approx$ iterations $\times$ per-iteration cost. In databases, the latter is dominated by join selectivity, degree distributions, and I/O characteristics of the edge table. Small design choices—precomputing out-degrees, deduplicating frontiers, avoiding self-loop blow-ups, and using composite indexes on edges(src,dst)—can shift plans from nested-loop patterns to columnar-friendly scan-join pipelines. My goal is not to “beat” specialized graph systems, but to identify the envelope where an SQL-first approach is good enough for day-to-day work, while still remaining simple, maintainable, and easy to integrate with surrounding relational processing.

B. Related Work

Languages and programmability. A first line of work raises the abstraction level so that developers can write graph logic without micromanaging joins. All-in-One extends SQL/Datalog with MM-join, MV-join, anti-join, and union-by-update, and shows how fixed-point recursion captures a wide class of graph algorithms with convergence guarantees [2]. A complementary path is to compile a graph-like surface to portable SQL. GRAPHiQL treats vertices, edges, neighborhoods, and loops as first-class concepts, so users think

in graph terms while the system emits optimized SQL for standard backends [3]. In my implementations, I consciously keep to plain SQL but borrow these ideas to keep the code structured and easy to audit.

Execution frameworks inside RDBMS. A second line studies how to execute graph search with set-at-a-time plans. BSEG introduces the FEM (Frontier-Select/Expand/Merge) pattern, SegTable (locally shortest segments), and bidirectional set-Dijkstra to reduce search space while exploiting bulk joins [4]. Follow-up work such as BRBFSw shows that delaying expansions, partitioning edge weights, and using window functions plus MERGE can reduce the product “iterations $\times$ per-iteration cost” in practice [5]. Beyond shortest paths, there is solid system evidence: SQL mappings on a columnar engine (Vertica) are competitive on iterative and 1-hop tasks [1]; and connected components can be solved via randomized contraction with expected $O(\log |V|)$ SQL rounds and linear space, often outperforming common big-data baselines [6]. These results make it reasonable for me to treat the RDBMS as a serious execution substrate, not a temporary workaround.

Database-as-a-graph search. A third thread views a relational database as a graph via foreign-key reachability and answers *structured keyword* and *community* queries. Representative methods include SQL-based tuple reduction followed by assembly for connected trees and reachable subgraphs [7], Top- k minimum-cost connected trees covering multiple keyword sets (GST- k) [8], and community queries that return multi-center, radius-bounded subgraphs with polynomial-delay enumeration and ranking [9]. Although my main focus is on foundational algorithms, this line gives useful design hints: stage aggressive pruning in SQL and then assemble results with predictable joins.

C. Contributions

This report is implementation-first. I keep the data model simple (normalized vertices/edges) and avoid vendor-specific UDFs, so the code is portable across systems.

- **SQL implementations of classic algorithms.** I provide working, fully relational implementations of *PageRank*, *HITS*, *Topological Sort*, *Connected Components*, and *Bellman-Ford (SSSP)*. Each follows a reproducible template: (i) clear schemas and keys; (ii) an iterative driver (recursive WITH or looped staging tables); and (iii) convergence/termination guards and idempotent updates. PageRank explicitly handles spider traps and dead ends by mass re-injection [2]; HITS alternates hub/authority updates with per-iteration ℓ_1 normalization; Connected Components uses label/BFS-style propagation with careful deduplication and is contrasted with randomized contraction [6]; Bellman-Ford is expressed as bulk relaxations over joins with early stopping; Topological Sort implements Kahn’s algorithm via iterative removal of zero in-degree nodes using anti-joins.
- **Two execution styles compared.** For several algorithms, I compare a declarative *recursive CTE* (fixed-point) style

with an *iterative staging-table* style. The former is concise and easy to read; the latter exposes control for delayed expansions, checkpointing, and batch updates.

- **Engineering guidelines.** I summarize choices that matter in practice: composite indexes on edges(src,dst), covering indexes for join keys, avoiding duplicate blow-ups from self-joins, frontier deduplication, and batching updates with MERGE or INSERT. . . ON CONFLICT. I also discuss memory-I/O trade-offs (temporary materialization vs. on-the-fly aggregation) and how columnar-friendly plans (scan-join-aggregate) naturally emerge [1].
- **Indicative measurements.** I report wall-clock runtimes on several datasets with different sizes and sparsities to sanity-check that the SQL implementations scale beyond toy examples. I only measure total runtime per dataset. I do not break down iteration counts, per-iteration costs, memory usage, or I/O statistics; such profiling is left for future work. The goal is practical: give a clear sense of whether the implementations are usable in common settings, not to build a full benchmark suite.

D. Organization

Section II reviews and implements the foundational algorithms above. Section III moves to in-DB tasks such as Connected Components, Keyword Search, and Communities. Section IV presents a language-system bridge using GRAPHiQL and discusses how a graph-intuitive surface compiles into the scan-join-aggregate backbone [2], [3]. I conclude with open problems around recursive plan costing, robust deduplication for high-degree hubs, and explainability for SQL-native graph results.

II. FOUNDATIONAL IN-DB ALGORITHMS

A. PageRank

After the rapid expansion of the World Wide Web, search engines faced two major challenges: whom to trust, and how to rank the most relevant results at the top. Since not all web pages are equally important, researchers proposed leveraging the link structure to organize and rank them.

The core intuition behind PageRank is to treat hyperlinks as votes: a page is considered more important if it is linked to by many other pages, especially by those that are themselves important—forming a recursive notion of importance.

PageRank is based on the “random surfer” model and defines the importance of a page (or more generally, a node in a graph) as the long-term probability that a random surfer lands on it. It has become a key signal in web page ranking and is also widely used as a centrality or influence measure in general graph analysis.

1) Concept (Definition and Intuition):

- **Random Surfer Model.** At each step, with probability β a surfer follows a random out-link; with probability $1 - \beta$ the surfer jumps uniformly to any page. Typical values are $\beta \in [0.8, 0.9]$.

- **PageRank Equation (Brin–Page, 1998).** For any page j ,

$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + \frac{1 - \beta}{N},$$

where d_i is the out-degree of i and N is the number of pages.

- **Google Matrix Form.** Let M be the column-stochastic transition matrix (normalized by out-degree). Then

$$A = \beta M + \frac{1 - \beta}{N} \mathbf{11}^\top, \quad r = Ar.$$

I.e. PageRank is the principal eigenvector of A .

- **Stationary Distribution View.** Equivalently, PageRank is the limiting stationary distribution of the random walk with teleports.

2) Spider Traps and Dead Ends (Problems and Fixes):

- **Dead Ends.** Pages with no out-links make the random walk “have nowhere to go” and cause rank to leak from the system.
- **Spider Traps.** A group whose out-links point only inside the group traps the walk and gradually absorbs all importance.
- **Why Teleports Fix This.** Spider traps bias scores; forcing teleports in finite steps prevents getting stuck. Dead ends break column-stochasticity; always teleporting when there is nowhere to go restores it.
- **Implementation Note.** Either preprocess M to remove dead ends or, at dead ends, follow random teleport links with probability 1.0.

3) *SQL Implementation:* Kaifeng Zhao et al. proposed a recursive implementation of the PageRank algorithm in their paper. Listing 1 illustrates the approach by Kaifeng Zhao et al. The recursive relation is expressed as $P(ID, W)$, with initialization occurring in line 3, where the W attribute is initially set to zero. The MV-join operation is implemented in lines 5–6, while the union-by-update logic is specified in line 4, where the ID attribute is used to determine whether two tuples are considered identical. The recursive relation P , introduced via the `WITH` clause, is ultimately produced by the SQL query shown in line 8.

Listing 2 presents my implementation of the iterative PageRank algorithm, which incorporates mechanisms to address spider traps and dead ends.

Listing 1 Recursive PageRank in SQL-like pseudocode (without handling dead ends & spider traps)

```

1  with
2  P(ID, W) as (
3    (select R.ID, 0.0 from R)
4    union by update ID
5    (select S.T, c * sum(W * ew) + (1 - c)/n from P, S
6     where P.ID = S.F group by S.T)
7    maxrecursion 10)
8  select ID, W from P

```

Graphs	V	E	Diameter	Avg. Degree	Time(s)
Youtube (YT)	1,134,890	2,987,624	20	5.27	168
LiveJournal (LJ)	3,997,962	34,681,189	17	17.35	1203
Orkut (OK)	3,072,441	117,185,083	9	76.22	7766

TABLE I

GRAPH STATISTICS AND PAGERANK RUNTIME ON POSTGRESQL

Listing 2 Iterative PageRank in SQL-like pseudocode

```

1  -- 1) Out-degree preprocessing (set out-degree to N for
   ↳ dangling nodes)
2  WITH outdeg AS (
3    SELECT
4      n.nodeid,
5      COALESCE(t.cnt, N) AS out_deg
6    FROM nodes n
7    LEFT JOIN (
8      SELECT e.sourcenodeid AS nodeid, COUNT(*) AS cnt
9      FROM edges e
10     GROUP BY e.sourcenodeid
11   ) t USING (nodeid)
12 )
13 UPDATE nodes AS n
14 SET nodecount = o.out_deg
15 FROM outdeg o
16 WHERE n.nodeid = o.nodeid;
17
18 -- 2) Iterative PageRank with teleport and dangling
   ↳ mass
19 -- Repeat until all nodes converged or max_iter reached
20 FOR iter FROM 1 TO max_iter LOOP
21   -- 2.1) Compute dangling mass (sum of ranks of nodes
   ↳ with no outgoing edge)
22   WITH dangling AS (
23     SELECT SUM(n.nodeweight) AS s
24     FROM nodes n
25     WHERE NOT EXISTS (
26       SELECT 1 FROM edges e WHERE e.sourcenodeid =
   ↳ n.nodeid
27     )
28   ),
29   base AS (
30     -- teleport term plus redistributed dangling mass
31     SELECT (1.0 - d) / N + d * COALESCE(s, 0.0) / N AS b
32     FROM dangling
33   ),
34   incoming AS (
35     -- sum of r[u] / outdeg[u] over all incoming
   ↳ neighbors u of each v
36     SELECT
37       e.targetnodeid AS nodeid,
38       SUM(nu.nodeweight / NULLIF(nu.nodecount, 0)) AS
   ↳ in_sum
39     FROM edges e
40     JOIN nodes nu ON nu.nodeid = e.sourcenodeid
41     GROUP BY e.targetnodeid
42   ),
43   new_weights AS (
44     SELECT
45       n.nodeid,
46       n.nodeweight AS old_w,
47       (SELECT b FROM base) + d * COALESCE(i.in_sum,
   ↳ 0.0) AS new_w
48     FROM nodes n
49     LEFT JOIN incoming i ON i.nodeid = n.nodeid
50   )
51 UPDATE nodes n
52 SET
53   nodeweight = nw.new_w,
54   hasconverged = ABS(nw.new_w - nw.old_w) < eps
55 FROM new_weights nw
56 WHERE n.nodeid = nw.nodeid;
57 -- 2.2) renormalize to keep sum of ranks = 1
58 WITH tot AS (SELECT SUM(nodeweight) AS s FROM nodes)
59 UPDATE nodes SET nodeweight = nodeweight / (SELECT s
   ↳ FROM tot);
60 -- 2.3) Stop if all nodes have converged
61 EXIT WHEN NOT EXISTS (SELECT 1 FROM nodes WHERE
   ↳ hasconverged = FALSE);
62
63 END LOOP;

```

B. HITS: Hubs and Authorities

HITS, short for Hypertext Induced Topic Selection, is a link analysis method proposed around the same time as PageRank in 1998. It assigns two complementary scores to every page. An authority score measures how valuable the page is as a source of information. A hub score measures how good the page is at pointing users to valuable sources. Links are treated as votes: a page becomes a strong authority when many good hubs link to it, and a page becomes a strong hub when it links to many strong authorities.

The algorithm starts with uniform scores for all pages. Then it repeatedly updates the two scores. The new authority score of a page is the sum of the hub scores of pages that link to it. The new hub score of a page is the sum of the authority scores of pages it links to. After each round the scores are normalized and the process continues until the values stabilize. In practice, authority pages are content rich home pages such as newspaper sites, while hub pages are directory like lists that coherently link to those authorities. HITS reveals coordinated endorsement patterns, not just raw link counts.

1) *Concept (Definition and Intuition)*: Treat hyperlinks as endorsements. Interesting pages fall into two classes:

- **Authorities**: content pages that many good hubs point to.
- **Hubs**: directory-like pages that point to many good authorities.

A good hub links to many good authorities; a good authority is linked from many good hubs. For each page i with hub score h_i and authority score a_i ,

$$a_i \propto \sum_{j \rightarrow i} h_j, \quad h_i \propto \sum_{i \rightarrow k} a_k.$$

Matrix view. Let A be the adjacency matrix with $A_{ij} = 1$ if $i \rightarrow j$. With vectors $h = (h_1, \dots, h_n)$ and $a = (a_1, \dots, a_n)$,

$$h \propto Aa, \quad a \propto A^T h,$$

so

h is the principal eigenvector of AA^T ,

a is the principal eigenvector of $A^T A$.

Intuition. Starting from uniform scores, repeatedly propagate authority to hubs and hubs to authorities, normalizing after each step, until the scores stabilize. The procedure uncovers coordinated endorsement patterns rather than counting links alone: hubs collect and recommend, while authorities provide high-quality content.

Graphs	V	E	Diameter	Avg. Degree	Time(s)
Youtube (YT)	1,134,890	2,987,624	20	5.27	454
LiveJournal (LJ)	3,997,962	34,681,189	17	17.35	1478
Orkut (OK)	3,072,441	117,185,083	9	76.22	17844

TABLE II

GRAPH STATISTICS AND HITS RUNTIME ON POSTGRESQL

2) *SQL Implementation*: Following pseudocode presents my implementation of the iterative HITS algorithm.

Listing 3 Iterative HITS in SQL-like pseudocode

```

1  SET margin := 1e-4;
2  SET diff   := +infinity;
3
4  WHILE diff >= margin LOOP
5    -- (1) backup current scores
6    UPDATE nodes
7    SET prev_auth = curr_auth,
8        prev_hub  = curr_hub;
9
10   -- (2) update authorities: sum of incoming hubs
11   WITH authority(nodeid, score) AS (
12     SELECT e.targetnodeid, SUM(COALESCE(n.curr_hub, 0))
13     FROM edges e
14     JOIN nodes n ON n.nodeid = e.sourcenodeid
15     GROUP BY e.targetnodeid
16   )
17   UPDATE nodes n
18   SET curr_auth = COALESCE(a.score, 0)
19   FROM authority a
20   WHERE n.nodeid = a.nodeid;
21
22   -- (3) normalize authorities to sum 1
23   s := (SELECT SUM(curr_auth) FROM nodes);
24   UPDATE nodes
25   SET curr_auth = CASE WHEN s = 0 THEN 0 ELSE curr_auth
26   ↪ / s END;
27
28   -- (4) update hubs: sum of outgoing authorities
29   WITH hub(nodeid, score) AS (
30     SELECT e.sourcenodeid, SUM(COALESCE(n.curr_auth,
31     ↪ 0))
32     FROM edges e
33     JOIN nodes n ON n.nodeid = e.targetnodeid
34     GROUP BY e.sourcenodeid
35   )
36   UPDATE nodes n
37   SET curr_hub = COALESCE(h.score, 0)
38   FROM hub h
39   WHERE n.nodeid = h.nodeid;
40
41   -- (5) normalize hubs to sum 1
42   t := (SELECT SUM(curr_hub) FROM nodes);
43   UPDATE nodes
44   SET curr_hub = CASE WHEN t = 0 THEN 0 ELSE curr_hub /
45   ↪ t END;
46
47   -- (6) convergence check (max L-infinity change)
48   diff := (
49     SELECT MAX(GREATEST(ABS(curr_auth - prev_auth),
50     ↪ ABS(curr_hub - prev_hub)))
51     FROM nodes
52   );
53   END LOOP;
54
55   -- final scores
56   SELECT nodeid, curr_auth AS authority, curr_hub AS hub
57   FROM nodes
58   ORDER BY nodeid;

```

Kaifeng Zhao et al. proposed a recursive implementation of the HITS algorithm in their paper.

Listing 4 Recursive HITS in SQL-like pseudocode

```

1  with
2  H(ID, h, a) as (
3    (select ID, 1.0, 1.0 from V)
4    union by update
5    (select ID, h / sqrt(nh), a / sqrt(na)
6     from R_ha, R_n)
7    computed by
8    H_h as select ID, h from H;
9
10  R_a(ID, a) as
11    select ID, sum(h * ew) from H_h, E
12    where H.ID = E.T
13    group by E.F;
14
15  R_h(ID, a) as
16    select ID, sum(a * ew) from R_a, E
17    where R_a.ID = E.F
18    group by E.T;
19
20  R_ha as
21    select R_a.ID, h, a from R_a, R_h
22    where R_a.ID = R_h.ID;
23
24  R_n(nh, na) as
25    select sum(h * h), sum(a * a)
26    from R_ha)
27  select * from H;

```

C. Topological Sort

Topological Sort is a linear ordering of the vertices in a directed acyclic graph (DAG) such that for every directed edge from vertex u to vertex v , u appears before v in the ordering. It is commonly used in scenarios involving dependency resolution, such as task scheduling, build systems, and course prerequisite planning. The algorithm can be implemented using either depth-first search (DFS) or Kahn's algorithm based on in-degree counting.

Listing 5 Recursive TopoSort in SQL-like pseudocode

```

1  with
2  Topo(ID, L) as (
3    (select ID, 0 from V
4     where ID not in (select E.T from E))
5    union all
6    (select ID, L from Tn)
7    computed by
8    Ln(L) as select max(L) + 1 from Topo;
9
10  V1 as
11    select V.ID from V
12    where ID not in (select ID from Topo);
13
14  E1 as
15    select E.F, E.T from V1, E
16    where V1.ID = E.F;
17
18  Tn as
19    select ID, L from V1, Ln
20    where ID not in (select T from E1);
21  )
22  select * from Topo;

```

Listing 6 Iterative TopoSort in SQL-like pseudocode

```

1  -- SQL-like pseudocode for Topological Sort using a
2  -- recursive CTE
3  -- Input tables: V(ID), E(F, T)
4
5  WITH RECURSIVE
6
7  -- 1) Precompute in-degrees for all vertices
8  in_degree(ID, indeg) AS (
9    SELECT v.ID, COUNT(e.F)
10   FROM V AS v
11   LEFT JOIN E AS e ON v.ID = e.T
12   GROUP BY v.ID
13 ),
14
15 -- 2) Build topological layers
16 Topo(ID, L, path) AS (
17   -- Base: all sources (indeg = 0) start at layer 0,
18   -- path = {ID}
19   SELECT ID, 0 AS L, ARRAY[ID] AS path
20   FROM in_degree
21   WHERE indeg = 0
22
23   UNION ALL
24
25   -- Step: from current node u, try to add neighbor v
26   SELECT e.T AS ID, t.L + 1 AS L, t.path || e.T AS path
27   FROM Topo AS t
28   JOIN E AS e ON e.F = t.ID
29   -- admit v only if all its predecessors are already
30   -- in t.path
31   WHERE NOT EXISTS (
32     SELECT 1
33     FROM E AS p
34     WHERE p.T = e.T -- predecessor p -> v
35           AND p.F <> ALL (t.path) -- some predecessor not
36           -- yet chosen
37   )
38 )
39
40 -- 3) Emit node with its layer (topological rank)
41 SELECT ID, L
42 FROM Topo
43 ORDER BY L;

```

D. Bellman-Ford(SSSP)

The Bellman-Ford algorithm is a single-source shortest path algorithm for graphs that may contain negative edge weights. It works by repeatedly relaxing all edges up to $V-1$ times, where V is the number of vertices. Unlike Dijkstra's algorithm, Bellman-Ford can handle negative weights and is able to detect negative-weight cycles. If any distance can still be reduced after $V-1$ iterations, a negative cycle exists.

Graphs	V	E	Diameter	Avg. Degree	Time(s)
Youtube (YT)	1,134,890	2,987,624	20	5.27	103
LiveJournal (LJ)	3,997,962	34,681,189	17	17.35	988
Orkut (OK)	3,072,441	117,185,083	9	76.22	4221

TABLE III

GRAPH STATISTICS AND BELLMAN-FORD RUNTIME ON POSTGRESQL

Listing 7 Bellman-Ford in SQL-like pseudocode

```

1
2 -- Input: start_vertex
3 -- Tables: vertices(id), edges(source, target, weight)
4
5 -- 0) Working table
6 CREATE TEMP TABLE dist (
7     vertex      INT PRIMARY KEY,
8     distance    NUMERIC,
9     predecessor INT
10 );
11
12 -- 1) Initialize distances
13 INSERT INTO dist(vertex, distance, predecessor)
14 SELECT v.id,
15        CASE WHEN v.id = start_vertex THEN 0 ELSE 1e18
16        ↪ END,
17        NULL
18 FROM vertices AS v;
19
20 -- 2) Relax edges |V|-1 times (early stop if no change)
21 FOR iter = 1 .. (SELECT COUNT(*) FROM vertices) - 1 LOOP
22     UPDATE dist d
23     SET distance    = s.distance + e.weight,
24         predecessor = e.source
25     FROM dist s
26     JOIN edges e ON e.source = s.vertex
27     WHERE e.target = d.vertex
28           AND s.distance + e.weight < d.distance;
29
30     -- if no rows updated in this iteration, break
31     IF ROW_COUNT() = 0 THEN EXIT LOOP; END IF;
32 END LOOP;
33
34 -- 3) Negative-cycle check
35 IF EXISTS (
36     SELECT 1
37     FROM dist d
38     JOIN edges e ON e.source = d.vertex
39     JOIN dist d2 ON d2.vertex = e.target
40     WHERE d.distance + e.weight < d2.distance
41 ) THEN
42     RAISE ERROR 'negative-weight cycle detected';
43 END IF;
44
45 -- 4) Result
46 SELECT vertex, distance, predecessor
47 FROM dist
48 ORDER BY vertex;

```

III. IN-DB SPECIALIZED ALGORITHMS

A. Connected Component

Connected-component analysis is a fundamental primitive for graph analytics, but most real-world, at-scale graphs live inside relational, SQL-accessible, Massively Parallel Processing (MPP) databases. Moving such data to a separate graph system incurs cost, risk, and duplicated pipelines; in-database execution avoids these drawbacks by running the analysis where the data already resides [6].

The paper [6] adopts Randomised Contraction as the core method for in-database connected components, following Bögeholz, Brand, and Todor’s *In-database connected component analysis*. Their algorithm is a linear-space, randomized approach that always returns correct components and, for any $\epsilon > 0$, terminates after $O(\log |V|)$ SQL rounds with probability at least $1 - \epsilon$; empirically this yields quasi-linear

end-to-end runtimes in practice. In addition, because each round processes exponentially less data, the overall query count is logarithmic and the work per round shrinks smoothly at scale.

The algorithm’s design is motivated by the limitations of two naïve in-database strategies. A breadth-first-style “propagate-the-minimum-ID” approach can require as many rounds as the graph diameter; on a path graph labelled $1 \dots n$, it needs $n-1$ rounds—untenable at Big Data scale. Conversely, repeatedly squaring the graph by joins $(G, G^2, G^4, \dots)$ blows the edge set up to $\Theta(|V|^2)$ in a single component, which is also impractical. Randomised Contraction achieves “the best of both worlds”: logarithmic-round progress while keeping storage linear in the input.

At a high level, each round maps every vertex to a representative within its closed neighborhood; edges are then “contracted” by replacing endpoints with their representatives, eliminating loops and duplicates. Repeating this process until only isolated vertices remain yields component labels for all original vertices. The result is an algorithm whose space footprint remains linear, with practical temporary storage bounded by a small constant multiple of the input.

Relative to the broader literature, Randomised Contraction offers a pragmatic bridge between theory and practice. On the PRAM model, optimal randomized algorithms achieve logarithmic time with high probability (e.g., Halperin and Zwick’s EREW PRAM algorithm), but the PRAM’s assumptions diverge from real parallel machines [10]. Recent large-scale systems work has therefore focused on MapReduce and Spark. Hash-to-Min (Rastogi et al.) attains $O(\log |V|)$ rounds but can require quadratic space in $|V|$ [11]; Two-Phase (Kiveris et al.) reduces space to linear but takes $\Theta(\log^2 |V|)$ rounds [12]; and the Spark-based Cracker (Lulli et al.) reaches $O(\log |V|)$ rounds at the cost of higher communication [13]. Randomised Contraction, implemented purely in SQL, matches or improves these trade-offs within an MPP database: it uses expected $O(\log |V|)$ rounds with linear space, and in head-to-head comparisons it consistently outperforms these alternatives across diverse, billion-scale datasets.

Finally, this in-database orientation integrates naturally with existing analytic ecosystems. Prior work like MADlib demonstrated the feasibility of machine learning and basic graph primitives inside SQL engines [14]; Randomised Contraction extends that story to an algorithmically robust, scalable connected-components solution that minimizes data movement and operational complexity.

Graphs	$ V $ (M)	$ E $ (M)	Components(k)	Time(s)
Andromeda	1,459	2,287	62,166	5431
Bitcoin address	878	830	216,917	1530
Candels10	83	238	39	424

TABLE IV
GRAPH STATISTICS AND CONNECTED COMPONENT RUNTIME ON
POSTGRESQL

Listing 8 Connected Component in SQL-like pseudocode

```

1  -- RANDOMISEDCONTRACTIONFAST(G)
2  -- Input: Edge table G(v, w)
3  -- Output: Result table containing connected components
4
5  -- Step 1: Symmetrize edge table
6  CREATE TABLE E AS
7  SELECT v, w FROM G
8  UNION ALL
9  SELECT w, v FROM G;
10
11 -- Step 2: Initialize
12 -- Use a stack S (implemented externally) to store (A,
13 -- B) pairs
14 -- Set i = 0
15
16 -- Step 3: Random contraction loop
17 REPEAT
18   SET i = i + 1;
19
20   -- Choose random affine parameters A, B
21   -- Push (A, B) to stack S
22
23   -- Compute representatives
24   CREATE TABLE R_i AS
25   SELECT
26     v,
27     LEAST(axb(A, v, B), MIN(axb(A, w, B))) AS r
28   FROM E
29   GROUP BY v;
30
31   -- Contract edges by representative mapping
32   CREATE TABLE T AS
33   SELECT DISTINCT
34     V.r AS v,
35     W.r AS w
36   FROM E
37   JOIN R_i AS V ON E.v = V.v
38   JOIN R_i AS W ON E.w = W.v
39   WHERE V.r <> W.r;
40
41   -- Check if graph is fully contracted
42   SET rowcount = number of rows in T;
43
44   DROP TABLE E;
45   ALTER TABLE T RENAME TO E;
46
47 UNTIL rowcount = 0;
48
49 -- Step 4: Backward composition of representative
50 -- functions
51 SET (A, B) = (1, 0);
52
53 WHILE i > 1 DO
54   SET i = i - 1;
55
56   -- Pop (alpha, beta) from stack S
57   SET (A, B) = (axb(A, alpha, 0), axb(A, beta, B));
58
59   -- Compose representative mapping
60   CREATE TABLE T AS
61   SELECT
62     L.v AS v,
63     COALESCE(R.r, axb(A, L.r, B)) AS r
64   FROM R_i AS L
65   LEFT OUTER JOIN R_{i+1} AS R
66   ON L.r = R.v;
67
68   DROP TABLE R_i;
69   DROP TABLE R_{i+1};
70   ALTER TABLE T RENAME TO R_i;
71 END WHILE;
72
73 -- Final result
74 ALTER TABLE R_1 RENAME TO Result;

```

B. Keyword Search

Keyword search over relational databases (RDBs) finds structural connections among tuples matching the terms, rather than ranking single tuples by text relevance. It reasons along foreign-key paths in the “database graph” to return connected substructures. In practice, this style of search helps non-experts explore complex schemas because each answer is a small evidence graph that explains why the tuples belong together. Instead of asking users to write joins, the system discovers and assembles plausible join patterns and presents them as readable trees or small subgraphs.

The paper [7] follows *Keyword Search in Databases: The Power of RDBMS*. Qin, Yu, and Chang show that modern engines can compute keyword search natively in SQL, without middleware or new indexes: first prune tuples that cannot appear in any answer (“tuple reduction”), then evaluate the remaining candidate networks by joins. In practice this means pushing simple filters and projections early, sharing common subexpressions, and avoiding large intermediate results before the final assembly. They unify three semantics: (i) connected trees with a size cap, (ii) distinct root—tuples reachable from a unique center within a radius, and (iii) distinct core—multi-center communities within a radius. The framework keeps data in place, reuses existing indexes and statistics, and exposes a clear knob for tuning result size and readability.

Two main lines of prior work frame the area. Schema-based systems enumerate candidate networks and evaluate them in SQL, as in DBXPLORER [15] and DISCOVER [16]. Graph-based systems materialize the database graph and search in memory, e.g., BANKS [17] and BLINKS [18]. The first family benefits from the optimizer but can face an explosion of candidate networks; the second offers flexible graph exploration but pays data-movement and freshness costs. The SQL-native path aims to take the best of both worlds: set-based execution with predictable plans, while avoiding a separate graph store.

The RDBMS-native alternative is: for connected trees, use a semijoin-then-join plan to shrink intermediates and share work across overlapping networks. For distinct root and distinct core, algorithms (DR and DC) compute distance relationships from potential centers to keyword tuples within a maximum radius $D_{\max}$, materialize compact relationship tables, and then assemble answers by grouping and joins.

Working “where the data lives” avoids data movement, keeps answers consistent with concurrent updates, and allows business filters to be pushed down (time windows, user segments, attribute constraints). It also aligns with community-style results and later work on querying communities in relational databases [19], while clarifying—alongside CN-based [15], [16] and graph-based systems [17], [18]—when in-database processing is preferable: when governance and freshness matter, when the schema provides useful structure for reduction, and when answers must be small, interpretable subgraphs rather than large result sets.

Listing 9 Keyword Search in SQL-like pseudocode

```

1  -- Inputs:
2  -- E(src, dst) -- foreign-key graph over the whole
   ↳ database (make it undirected)
3  -- KW(tid, kid) -- tuples that match keyword kid
   ↳ {1..M}
4  -- Parameters:
5  -- DMAX          -- radius cap (integer)
6  -- M             -- number of keywords in the query
   ↳ (integer)
7
8  -- Multi-source BFS from every keyword hit
   ↳ (simultaneous expansion up to DMAX)
9  WITH RECURSIVE R(kid, center, tid, d) AS (
10     -- layer 0: each keyword hit is its own center
11     SELECT k.kid, k.tid AS center, k.tid AS tid, 0 AS d
12     FROM KW AS k
13     UNION ALL
14     -- expand along FK edges; stop when reaching DMAX
15     SELECT r.kid, r.center, e.dst, r.d + 1
16     FROM R AS r
17     JOIN E AS e ON e.src = r.tid
18     WHERE r.d < DMAX
19 ),
20 -- Keep shortest distance per (center, kid, tid) to
   ↳ control growth
21 MD AS (
22     SELECT kid, center, tid, MIN(d) AS d
23     FROM R
24     GROUP BY kid, center, tid
25 ),
26 -- Candidate centers: must reach ALL M keywords within
   ↳ DMAX
27 C AS (
28     SELECT center
29     FROM MD
30     WHERE d <= DMAX
31     GROUP BY center
32     HAVING COUNT(DISTINCT kid) = M
33 )
34
35 -- ===== Distinct-Root (DR): one closest tuple per
   ↳ keyword for each center =====
36 SELECT center, kid, tid, d
37 FROM (
38     SELECT
39         m.*,
40         ROW_NUMBER() OVER (PARTITION BY center, kid ORDER
   ↳ BY d, tid) AS rn
41     FROM MD AS m
42     JOIN C AS c ON c.center = m.center
43     WHERE m.d <= DMAX
44 ) AS ranked
45 WHERE rn = 1;
46
47 -- ===== Distinct-Core (DC): group centers by the
   ↳ chosen tuple-ids (one per kid) =====
48 WITH DR AS (
49     SELECT center, kid, tid, d
50     FROM (
51         SELECT
52             m.*,
53             ROW_NUMBER() OVER (PARTITION BY center, kid ORDER
   ↳ BY d, tid) AS rn
54         FROM MD AS m
55         JOIN C AS c ON c.center = m.center
56         WHERE m.d <= DMAX
57     ) AS ranked
58     WHERE rn = 1
59 )
60 SELECT
61     d1.tid AS tid_k1,
62     d2.tid AS tid_k2,
63     ARRAY_AGG(DISTINCT d1.center) AS centers -- use
   ↳ LIST/JSON/ARRAY as your DB supports
64 FROM DR AS d1
65 JOIN DR AS d2
66     ON d1.center = d2.center AND d1.kid = 1 AND d2.kid = 2
67 GROUP BY d1.tid, d2.tid;

```

C. Querying Communities

Keyword search over relational data is often implemented as finding minimal connected trees that together cover all query terms. While tree answers are intuitive, they can split evidence across many small structures and hide how keyword-bearing tuples relate through multiple hubs and alternative paths. Querying Communities in Relational Databases reframes the task as discovering communities: radius-bounded, multi-center induced subgraphs that contain all keyword tuples and the shortest paths connecting them. In this model, a result is defined by its core (the chosen keyword tuples); once the core is fixed, the centers and path nodes are determined by shortest-path constraints within the radius. The radius becomes a simple control knob: smaller values emphasize tight, highly cohesive evidence; larger values allow the community to reveal cross-links that trees would scatter across many separate answers. Because the output is an induced subgraph, users see every relevant edge among the participating tuples, which makes explanations clearer and reduces the need to browse dozens of near-duplicate trees.

The paper contributes two practical enumeration algorithms with polynomial delay. COMM-all lists every community without duplicates, and COMM-k returns the exact top-k communities under a distance-based cost that favors compact, well-connected results. “Polynomial delay” means each additional answer arrives after work that is polynomial in the input size, so the system remains responsive as results stream out. Duplicate control is handled by treating the core as a unique identifier, avoiding costly graph-isomorphism checks. For ranking, the framework is agnostic to the exact scoring function; the distance-based objective is used because it is simple, monotone, and interpretable. An interactive k feature lets users increase k and continue enumeration without restarting, which fits exploratory search. To scale to large databases, the authors introduce an index-driven graph projection: inverted lists identify candidate keyword nodes and nearby edges, and a neighborhood filter keeps only nodes that could serve as centers or lie on center-to-keyword paths within the radius. The projected graph is a small fraction of the original, yet it preserves answers exactly, which cuts both memory and runtime. Experiments on DBLP and IMDB show that these methods are efficient and return informative results.

This community perspective complements two main lines of prior work. Tree-based systems such as DISCOVER focus on generating and ranking minimal trees inside the DBMS [16]. Graph browsing and ranking systems such as BANKS expand from multiple sources on the database graph to support interactive exploration [17]. The community formulation also connects to distance-based top-k objectives studied in keyword proximity search [20]. By producing a single induced subgraph that captures multiple hubs and alternative connections, the approach provides a concise, comprehensive view that users can understand without sifting through many separate trees.

Listing 10 Querying Communities in SQL-like pseudocode

```

1  -- Inputs:
2  -- GD(V, E, w)    -- database graph: tuples as nodes
3  -- V, FK edges E with weights w
4  -- KW_i(tid)      -- nodes that contain keyword i,
5  -- for i = 1..L
6  -- RMAX           -- radius threshold
7  -- Output:
8  -- communities R  -- induced subgraphs for all (or
9  -- top-k) cores
10
11 -- 1) Neighbor sets (candidates that can serve as
12 -- centers for keyword i)
13 CREATE TEMP TABLE N_i AS
14 SELECT u AS node
15 FROM DIJKSTRA_REVERSE(      -- run Dijkstra on
16 -- graph with edges reversed
17 -- ADD_SINK_EDGES(GD, KW_i)) -- add zero-weight
18 -- edges (v -> t) for v in KW_i
19 WHERE dist_to_sink <= RMAX; -- i = 1..L
20 -- (Repeat to build N_1..N_L; Ni construction per
21 -- Algorithm 2.)
22
23 -- 2) Best core (choose one node per keyword nearest to
24 -- a common center; minimize cost)
25 WITH N AS (
26   SELECT node FROM N_1
27   INTERSECT SELECT node FROM N_2
28   -- ...
29   INTERSECT SELECT node FROM N_L
30 ),
31 nearest AS ( -- precomputed table keyed by (center,u,
32 -- kid) with (v_i, dist)
33   SELECT center, kid, kw_node AS v_i, d
34   FROM PRECOMPUTED_NEAREST      -- built while
35   -- computing Ni
36 ),
37 costs AS (
38   SELECT center,
39   JSON_OBJECT_AGG(kid, v_i) AS core_map,
40   SUM(d) AS cost
41   FROM nearest
42   GROUP BY center
43 )
44 SELECT core_map, cost
45 INTO BEST_CORE
46 FROM costs
47 ORDER BY cost, center
48 FETCH FIRST 1 ROW ONLY;      -- returns C and
49 -- its cost
50
51 -- 3) GetCommunity(C): build the induced community for
52 -- core C
53 CREATE TEMP TABLE Vc AS
54 SELECT u
55 FROM COVERAGE_BY_KNODES(C) -- run Dijkstra from each
56 -- knode in C; keep u that reach all knodes within RMAX
57 WHERE covered_keywords = L;
58
59 CREATE TEMP TABLE D_s AS -- distances from virtual
60 -- source s
61 SELECT u, dist_s
62 FROM DIJKSTRA_FROM_SOURCE(ADD_SOURCE_EDGES(GD, Vc));
63
64 CREATE TEMP TABLE D_t AS -- distances to virtual sink
65 -- t (reverse run)
66 SELECT u, dist_t
67 FROM DIJKSTRA_TO_SINK(ADD_SINK_EDGES(GD,
68 -- CORE_NODES(C)));
69
70 CREATE TEMP TABLE V_R AS
71 SELECT u
72 FROM D_s JOIN D_t USING(u)
73 WHERE dist_s + dist_t <= RMAX;
74
75 CREATE TABLE COMMUNITY AS
76 SELECT *
77 FROM INDUCE_SUBGRAPH(GD, V_R);
78

```

D. Relational bi-directional selective path search on SegTable (SPSP)

The BSEG line of work asks a practical question: can shortest-path queries be executed efficiently inside a relational database without exporting the graph to a separate engine? The framework starts from a Dijkstra-style baseline for weighted graphs and then pushes the computation into set-at-a-time SQL plans rather than node-at-a-time pointer chasing. Concretely, the authors introduce an iterative pattern with three steps—select the active frontier, expand it through the edge table, and merge improvements—which maps cleanly to scans, joins, and grouped updates. This mapping allows the optimizer to leverage indexes, compression, and window/merge features while keeping all data in place.

BSEG’s distinctive contribution is SegTable: a precomputed table of locally shortest “segments” (e.g., two-hop pieces) that can be stitched together during search. The paper also explores bidirectional search and constraints that stop exploration when forward and backward waves are close enough to certify optimality—again expressed in SQL with grouping and filtering predicates over the visited sets. These choices are driven by a simple observation: most time is spent in the expand step, so any technique that shrinks the number of expands or the size of intermediate joins can pay off.

The work positions itself against two families of alternatives. First, external or index-heavy solutions precompute labels or access structures to answer distance/path queries quickly; these include two-hop/labeling-style methods and specialized shortest-path indexes [21], [22]. Such methods can be very fast at query time but impose substantial build and maintenance cost, which is undesirable when graphs evolve or when queries must interleave with relational filters. Second, approximate distance estimation can guide or replace search, but it trades accuracy for speed and still sits outside the transactional RDBMS workflow [23]. By contrast, BSEG keeps everything inside SQL, letting administrators reuse operational safeguards and hardware already devoted to the database.

Finally, the paper acknowledges that the relational approach benefits from broader system trends. The move toward scalable data-parallel processing and bulk operators showed the value of batching and scan-heavy plans even for iterative workloads [24]. BSEG exploits the same idea in a single RDBMS by organizing the search into a few large, predictable statements and leaning on the optimizer. In practice, BSEG works best when SegTable does not explode in size (e.g., graphs with moderate diameter or when thresholds limit segment creation). When segments would be too many, the authors’ set-at-a-time Dijkstra and bidirectional variants remain viable fallbacks, preserving the “stay in SQL” principle while keeping performance competitive with specialized systems.

Graphs	V	E	Time(s) (lthd=2)	Time(s) (lthd=6)	Time(s) (lthd=10)
DBLP	312,967	1,149,663	3.9	1.9	1.8
GoogleWeb	855,802	5,066,842	3.9	3.6	5.5

TABLE V

GRAPH STATISTICS AND BSEG RUNTIME ON POSTGRESQL

Listing 11 BSEG in SQL-like pseudocode

```

1  -- Bi-directional Selective Search on SegTable (short
2  ↪ SQL-like pseudocode)
3
4  -- init
5  INSERT INTO TVisited(nid,d2s,p2s,f)
6  ↪ VALUES('s',0,'s',0);
7  INSERT INTO TVisited(nid,d2t,p2t,b)
8  ↪ VALUES('t',0,'t',0);
9  -- scalars (conceptual): minCost:=+inf; lf:=0; lb:=0;
10 ↪ nf:=1; nb:=1; fwd:=1; bwd:=1; Max:=999999;
11
12 WHILE (lb+lf <= minCost) AND nf>0 AND nb>0 LOOP
13   IF nf <= nb THEN
14     -- FORWARD round
15     UPDATE TVisited SET f=2
16     WHERE f=0 AND (d2s <= fwd*lfthd OR d2s=(SELECT
17       ↪ MIN(d2s) FROM TVisited WHERE f=0));
18
19     MERGE INTO TVisited AS trg
20     USING (
21       SELECT nid, p2s, cost FROM (
22         SELECT out.tid AS nid, out.pid AS p2s,
23         ↪ out.cost+q.d2s AS cost,
24         ROW_NUMBER() OVER(PARTITION BY out.tid
25         ↪ ORDER BY out.cost+q.d2s) rk
26       FROM TVisited q JOIN TOutSegs out ON
27       ↪ q.nid=out.fid
28       WHERE q.f=2 AND out.cost+q.d2s+lf < minCost
29       ) x WHERE rk=1
30     ) AS src(nid,p2s,cost)
31     ON src.nid=trg.nid
32     WHEN MATCHED AND trg.d2s>src.cost THEN
33       UPDATE SET d2s=src.cost, p2s=src.p2s, f=0
34     WHEN NOT MATCHED THEN
35       INSERT(nid,d2s,d2t,p2s,f)
36       ↪ VALUES(src.nid,src.cost,Max,src.p2s,0);
37
38     UPDATE TVisited SET f=1 WHERE f=2;
39     SELECT MIN(d2s) INTO lf FROM TVisited WHERE f=0;
40     fwd := fwd + 1;
41   ELSE
42     -- BACKWARD round
43     UPDATE TVisited SET b=2
44     WHERE b=0 AND (d2t <= bwd*lfthd OR d2t=(SELECT
45       ↪ MIN(d2t) FROM TVisited WHERE b=0));
46
47     MERGE INTO TVisited AS trg
48     USING (
49       SELECT nid, p2t, cost FROM (
50         SELECT _in.tid AS nid, _in.pid AS p2t,
51         ↪ _in.cost+q.d2t AS cost,
52         ROW_NUMBER() OVER(PARTITION BY _in.tid
53         ↪ ORDER BY _in.cost+q.d2t) rk
54       FROM TVisited q JOIN TInSegs _in ON
55       ↪ q.nid=_in.fid
56       WHERE q.b=2 AND _in.cost+q.d2t+lf < minCost
57       ) x WHERE rk=1
58     ) AS src(nid,p2t,cost)
59     ON src.nid=trg.nid
60     WHEN MATCHED AND trg.d2t>src.cost THEN
61       UPDATE SET d2t=src.cost, p2t=src.p2t, b=0
62     WHEN NOT MATCHED THEN
63       INSERT(nid,d2s,d2t,p2t,b)
64       ↪ VALUES(src.nid,Max,src.cost,src.p2t,0);
65
66     UPDATE TVisited SET b=1 WHERE b=2;
67     SELECT MIN(d2t) INTO lb FROM TVisited WHERE b=0;
68     bwd := bwd + 1;
69   END IF;
70
71   SELECT MIN(d2s+d2t) INTO minCost FROM TVisited; --
72   ↪ global best so far
73 END LOOP;
74
75 -- meeting node on the shortest path
76 SELECT nid INTO xid FROM TVisited WHERE d2s+d2t=minCost
77 ↪ LIMIT 1;

```

IV. A SYSTEM BLUEPRINT FOR IN-DATABASE GRAPH ANALYTICS WITH GRAPHiQL

GRAPHiQL asks a practical question many teams face: if graph data already live in a relational database, can we analyze them there without moving to a separate engine. The paper answers yes, but only if we raise the programming surface and compile down to SQL. The motivation is straightforward. Vertex centric systems offer an intuitive mental model yet require procedural code and feel awkward for neighborhood heavy or global tasks such as triangle counting and one hop statistics [25]–[27]. Raw SQL on node and edge tables is powerful but painful. Even basic logic explodes into long chains of self joins that are hard to write, debug, and tune. Writing pure SQL for PageRank, shortest paths, or triangles is especially hard and error prone. That difficulty is the direct reason GRAPHiQL exists.

The language introduces a graph table abstraction that holds nodes, edges, and properties, plus simple accessors for outgoing and incoming neighbors. On top of this, GRAPHiQL offers a small set of orthogonal operators. Iterate lets users write foreach or while loops. Filter selects a subgraph by predicates over nodes or edges. Retrieve projects the attributes that matter. Update changes values under conditions. Aggregate applies count, sum, min, max, or average with implicit grouping. These operators nest naturally so programs read like walk a neighborhood, then reduce. PageRank becomes a short loop that distributes mass over out neighbors and recollects it with an aggregate. Single source shortest path takes the minimum of neighbor distances until no value changes. Triangle counting chains two neighborhood steps without hand writing three self joins.

The evaluation compares compiled GRAPHiQL programs with a vertex centric engine on PageRank, single source shortest path, global and local triangle counting, strong overlap, and weak ties. On small graphs GRAPHiQL is faster because there is no distributed startup or message passing. On larger graphs it remains faster for PageRank and competitive for single source shortest path, while neighborhood heavy tasks that strain vertex centric engines continue to run as optimized joins inside the database [25]–[27]. Results on a column store show that a modern RDBMS executes these plans efficiently and that staying in one engine avoids data movement and governance costs [28]. Positioned against prior art, GRAPHiQL sits between vertex centric systems [25]–[27], [29], RDBMS case studies that report strong triangle analytics inside a column store [28], and query optimization work for graph shaped patterns [30]. The contribution is not universal speedups, but a way to make common graph tasks easy to express and fast enough where the data already live.

Listing 12 PageRank(GRAPHiQL style)

```

1  -- Init
2  FOREACH n IN g(type=N) SET n.value = 1.0 /
   ↳ COUNT(g(type=N))
3  FOREACH n IN g(type=N) SET n.outdeg =
   ↳ COUNT(n.out(type=N))
4  FOREACH n IN g(type=N) WHERE n.outdeg == 0 SET n.outdeg
   ↳ = COUNT(g(type=N)) -- treat dangling
5
6  FOREACH i IN [1, max_iter]
7    base = (1 - d) / COUNT(g(type=N))
8    + d * SUM(FOREACH z IN g(type=N)
9              WHERE COUNT(z.out(type=N)) == 0
10             GET z.value) / COUNT(g(type=N))
11
12   FOREACH n IN g(type=N)
13     in_sum = SUM(FOREACH u IN n.in(type=N) GET u.value
14               ↳ / u.outdeg)
15     n.next = base + d * in_sum
16
17   FOREACH n IN g(type=N) SET n.value = n.next
18 END

```

B. HITS in GRAPHiQL**Listing 13** HITS(GRAPHiQL style)

```

1  -- Assumptions: graph table g with node props: n.auth,
   ↳ n.hub, n.prev_auth, n.prev_hub
2  -- Params: margin, max_iter
3
4  -- (optional) init
5  FOREACH n IN g(type=N) SET n.auth = 1.0
6  FOREACH n IN g(type=N) SET n.hub = 1.0
7
8  diff = +INF
9  iter = 0
10 WHILE diff >= margin AND iter < max_iter
11   iter = iter + 1
12
13   -- backup
14   FOREACH n IN g(type=N) SET n.prev_auth = n.auth,
15   ↳ n.prev_hub = n.hub
16
17   -- authorities: a(v) =  $\sum_{u \rightarrow v} \text{hub}(u)$ 
18   FOREACH n IN g(type=N)
19     n.auth = SUM(FOREACH u IN n.in(type=N) GET u.hub)
20
21   s = SUM(FOREACH n IN g(type=N) GET n.auth)
22   FOREACH n IN g(type=N) SET n.auth = (s == 0 ? 0 :
23   ↳ n.auth / s)
24
25   -- hubs: h(u) =  $\sum_{u \rightarrow v} \text{auth}(v)$ 
26   FOREACH n IN g(type=N)
27     n.hub = SUM(FOREACH v IN n.out(type=N) GET v.auth)
28
29   t = SUM(FOREACH n IN g(type=N) GET n.hub)
30   FOREACH n IN g(type=N) SET n.hub = (t == 0 ? 0 :
31   ↳ n.hub / t)
32
33   -- convergence
34   diff = MAX(
35     FOREACH n IN g(type=N)
36       GET MAX(ABS(n.auth - n.prev_auth), ABS(n.hub
37       ↳ - n.prev_hub))
38   )
39 END

```

Listing 14 Bellman-Ford(GRAPHiQL style)

```

1
2  -- Inputs: start (source id), graph table g with:
3  --   nodes: g(type=N) having n.id, n.dist, n.pred
4  --   edges: g(type=E) having e.src (node), e.dst
   ↳ (node), e.w (weight)
5
6  INF = 1e18
7  V = COUNT(g(type=N))
8
9  -- init
10 FOREACH n IN g(type=N)
11   n.dist = (n.id == start ? 0 : INF)
12   n.pred = NULL
13
14  i = 0
15  updates = 1
16  WHILE i < V - 1 AND updates > 0
17   i = i + 1
18
19   -- stage relaxations
20   FOREACH v IN g(type=N)
21     v.cand = MIN(FOREACH e IN v.in(type=E) GET
22               ↳ e.src.dist + e.w)
23     v.cand_pred = FIRST(FOREACH e IN v.in(type=E)
24                       WHERE e.src.dist + e.w == v.cand
25                       GET e.src.id)
26
27   updates = COUNT(
28     FOREACH v IN g(type=N)
29       WHERE v.cand IS NOT NULL AND v.cand < v.dist
30       GET 1
31   )
32
33   FOREACH v IN g(type=N)
34     WHERE v.cand IS NOT NULL AND v.cand < v.dist
35     SET v.dist = v.cand, v.pred = v.cand_pred
36   END
37
38   -- negative-cycle check (one more relaxation would
   ↳ improve)
39   neg = COUNT(
40     FOREACH v IN g(type=N)
41       GET (MIN(FOREACH e IN v.in(type=E) GET e.src.dist +
42       ↳ e.w) < v.dist ? 1 : 0)
43   )
44   IF neg > 0 RAISE "negative-weight cycle"

```

REFERENCES

- [1] A. Jindal, S. Madden, M. Castellanos, and M. Hsu, “Graph analytics using vertical relational database,” in *2015 IEEE International Conference on Big Data (Big Data)*, 2015, pp. 1191–1200.
- [2] K. Zhao and J. X. Yu, “All-in-one: Graph processing in rdbms revisited,” in *Proceedings of the 2017 ACM International Conference on Management of Data*, ser. SIGMOD ’17. New York, NY, USA: Association for Computing Machinery, 2017, p. 1165–1180. [Online]. Available: <https://doi.org/10.1145/3035918.3035943>
- [3] A. Jindal and S. Madden, “Graphiq: A graph intuitive query language for relational databases,” in *2014 IEEE International Conference on Big Data (Big Data)*, 2014, pp. 441–450.
- [4] J. Gao, R. Jin, J. Zhou, J. X. Yu, X. Jiang, and T. Wang, “Relational approach for shortest path discovery over large graphs,” 2011. [Online]. Available: <https://arxiv.org/abs/1201.0232>
- [5] J. Gao, J. Zhou, J. X. Yu, and T. Wang, “Shortest path computing in relational dbms,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 4, pp. 997–1011, 2014.
- [6] H. Bögeholz, M. Brand, and R.-A. Todor, “In-database connected component analysis,” in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, 2020, pp. 1525–1536.
- [7] L. Qin, J. X. Yu, and L. Chang, “Keyword search in databases: the power of rdbms,” in *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’09. New York, NY, USA: Association for Computing Machinery, 2009, p. 681–694. [Online]. Available: <https://doi.org/10.1145/1559845.1559917>
- [8] B. Ding, J. Xu Yu, S. Wang, L. Qin, X. Zhang, and X. Lin, “Finding top-k min-cost connected trees in databases,” in *2007 IEEE 23rd International Conference on Data Engineering*, 2007, pp. 836–845.
- [9] L. Qin, J. X. Yu, L. Chang, and Y. Tao, “Querying communities in relational databases,” in *2009 IEEE 25th International Conference on Data Engineering*, 2009, pp. 724–735.
- [10] S. Halperin and U. Zwick, “An optimal randomized logarithmic time connectivity algorithm for the crew pram (extended abstract),” in *Proceedings of the Sixth Annual ACM Symposium on Parallel Algorithms and Architectures*, ser. SPAA ’94. New York, NY, USA: Association for Computing Machinery, 1994, p. 1–10. [Online]. Available: <https://doi.org/10.1145/181014.181017>
- [11] V. Rastogi, A. Machanavajjhala, L. Chitnis, and A. D. Sarma, “Finding connected components on map-reduce in logarithmic rounds,” 2012. [Online]. Available: <https://arxiv.org/abs/1203.5387>
- [12] R. Kiveris, S. Lattanzi, V. Mirrokni, V. Rastogi, and S. Vassilvitskii, “Connected components in mapreduce and beyond,” in *Proceedings of the ACM Symposium on Cloud Computing*, ser. SOCC ’14. New York, NY, USA: Association for Computing Machinery, 2014, p. 1–13. [Online]. Available: <https://doi.org/10.1145/2670979.2670997>
- [13] A. Lulli, E. Carlini, P. Dazzi, C. Lucchese, and L. Ricci, “Fast connected components computation in large graphs by vertex pruning,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 3, p. 760–773, Mar. 2017. [Online]. Available: <https://doi.org/10.1109/TPDS.2016.2591038>
- [14] J. Hellerstein, C. Ré, F. Schoppmann, D. Z. Wang, E. Fratkin, A. Gorajek, K. S. Ng, C. Welton, X. Feng, K. Li, and A. Kumar, “The madlib analytics library or mad skills, the sql,” 2012. [Online]. Available: <https://arxiv.org/abs/1208.4165>
- [15] S. Agrawal, S. Chaudhuri, and G. Das, “Dbxplorer: enabling keyword search over relational databases,” in *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’02. New York, NY, USA: Association for Computing Machinery, 2002, p. 627. [Online]. Available: <https://doi.org/10.1145/564691.564782>
- [16] V. Hristidis and Y. Papakonstantinou, “Discover: keyword search in relational databases,” in *Proceedings of the 28th International Conference on Very Large Data Bases*, ser. VLDB ’02. VLDB Endowment, 2002, p. 670–681.
- [17] G. Bhalotia, A. Hulgeri, C. Nakhe, S. Chakrabarti, and S. Sudarshan, “Keyword searching and browsing in databases using banks,” 02 2002, pp. 431–440.
- [18] H. He, H. Wang, J. Yang, and P. S. Yu, “Blinks: ranked keyword searches on graphs,” in *Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’07. New York, NY, USA: Association for Computing Machinery, 2007, p. 305–316. [Online]. Available: <https://doi.org/10.1145/1247480.1247516>
- [19] L. Qin, J. X. Yu, L. Chang, and Y. Tao, “Querying communities in relational databases,” in *Proceedings of the 2009 IEEE International Conference on Data Engineering*, ser. ICDE ’09. USA: IEEE Computer Society, 2009, p. 724–735. [Online]. Available: <https://doi.org/10.1109/ICDE.2009.67>
- [20] B. Kimelfeld and Y. Sagiv, “Finding and approximating top-k answers in keyword proximity search,” in *Proceedings of the Twenty-Fifth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, ser. PODS ’06. New York, NY, USA: Association for Computing Machinery, 2006, p. 173–182. [Online]. Available: <https://doi.org/10.1145/1142351.1142377>
- [21] S. Trißl and U. Leser, “Fast and practical indexing and querying of very large graphs,” in *Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’07. New York, NY, USA: Association for Computing Machinery, 2007, p. 845–856. [Online]. Available: <https://doi.org/10.1145/1247480.1247573>
- [22] F. Wei, “Tedi: efficient shortest path query answering on graphs,” in *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’10. New York, NY, USA: Association for Computing Machinery, 2010, p. 99–110. [Online]. Available: <https://doi.org/10.1145/1807167.1807181>
- [23] M. Potamias, F. Bonchi, C. Castillo, and A. Gionis, “Fast shortest path distance estimation in large networks,” in *Proceedings of the 18th ACM Conference on Information and Knowledge Management*, ser. CIKM ’09. New York, NY, USA: Association for Computing Machinery, 2009, p. 867–876. [Online]. Available: <https://doi.org/10.1145/1645953.1646063>
- [24] J. Dean and S. Ghemawat, “Mapreduce: simplified data processing on large clusters,” *Commun. ACM*, vol. 51, no. 1, p. 107–113, Jan. 2008. [Online]. Available: <https://doi.org/10.1145/1327452.1327492>
- [25] G. Malewicz, M. H. Austern, A. J. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski, “Pregel: a system for large-scale graph processing,” in *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’10. New York, NY, USA: Association for Computing Machinery, 2010, p. 135–146. [Online]. Available: <https://doi.org/10.1145/1807167.1807184>
- [26] Y. Low, D. Bickson, J. Gonzalez, C. Guestrin, A. Kyrola, and J. M. Hellerstein, “Distributed graphlab: a framework for machine learning and data mining in the cloud,” *Proc. VLDB Endow.*, vol. 5, no. 8, p. 716–727, Apr. 2012. [Online]. Available: <https://doi.org/10.14778/2212351.2212354>
- [27] J. E. Gonzalez, Y. Low, H. Gu, D. Bickson, and C. Guestrin, “PowerGraph: Distributed Graph-Parallel computation on natural graphs,” in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. Hollywood, CA: USENIX Association, Oct. 2012, pp. 17–30. [Online]. Available: <https://www.usenix.org/conference/osdi12/technical-sessions/presentation/gonzalez>
- [28] S. Lawande, L. Shrinivas, R. Venkatesh, and S. Walkauskas, “Scalable social graph analytics using the vertica analytic platform,” in *Enabling Real-Time Business Intelligence*, M. Castellanos, U. Dayal, and W. Lehner, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 98–108.
- [29] Apache Software Foundation, “Apache giraph: Project page,” <https://giraph.apache.org/>, 2025, accessed: 2025-08-09.
- [30] J. Huang, K. Venkatraman, and D. J. Abadi, “Query optimization of distributed pattern matching,” in *2014 IEEE 30th International Conference on Data Engineering*, 2014, pp. 64–75.